

Security Guide

Pandoras Box -- User Documentation

Version 1.0 | pandoras-box.ai

Security Guide

Version: 1.0 **Audience:** System operators and anyone who wants to understand the security model

Disclaimer

This software is provided under the MIT License on an "as-is" basis, without warranty. You are responsible for securing the machine it runs on, rotating credentials, maintaining access controls, and ensuring compliance with applicable laws. Nothing here constitutes professional security or legal advice.

1. Design Philosophy

The core security principle of this system is: ****no agent should be able to do more than its defined role, and no agent should be trusted by default.****

Every action is reviewed before it executes. Every agent operates in isolation from every other. No component holds credentials it does not need. And an independent, uninstrutable oversight process watches everything.

This is not a security afterthought -- it is the architecture.

2. Four-Tier Architecture

The system is divided into four tiers. A lower-numbered tier cannot instruct a higher-numbered tier. This is enforced structurally, not by policy.

Tier 0: System Administrator

CLI-based admin. No external connectivity.
Manages infrastructure. Talks only to you.

Tier 1: Security Overseer

Independent daemon. Approves or blocks every job.
Cannot be instructed by any agent or conductor.
No company credentials. No API access.

Tier 2: Conductors (one per company)

Receive inbound messages. Route jobs to the queue.
Hold no company credentials. Routing only.

Tier 3: Task Agents (per company: mail, calendar, files, voice)

Execute approved jobs. Scoped credentials only.
OS-isolated per company. Cannot call tools outside their scope.

Why this matters: Even if an agent were compromised (via a prompt injection in an email, for example), it cannot escalate to the Security Overseer tier, access another company's data, or take actions not defined in its scope. The damage is contained.

3. OS-Level Tenant Isolation

Each company runs under a dedicated macOS service account. The file system enforces the isolation at the OS level -- not just in code.

Control	Value
Service account	One per company (separate macOS user)
Directory permissions	750 (owner: service account, inaccessible to others)

Control	Value
Credential files	600 (readable only by the owning service account)
Token cache files	640 (readable by service account and admin group)
Job queue records	Partitioned by <code>company_id</code> ; agents only query their own records

What this prevents:

- The mail agent for Company A cannot read Company B's credentials even if it tries
- A compromised Company A agent cannot access Company B's email or files
- No shared memory between company processes -- each is a separate Node.js process

4. The Security Overseer

The Security Overseer is the most important security component. It is a standalone daemon running under its own macOS service account. It cannot be instructed by any agent, conductor, or user message. The only entity that can interact with it is the System Administrator -- and only by reviewing its output and running pre-approved scripts.

What it checks for every job:

- **Job type allowlist:** only known, expected job types are approved. An unexpected job type is blocked regardless of who requested it.
- **Payload inspection:** the job's parameters are checked for anomalous content, including patterns associated with instruction override attempts.
- **Rate limiting:** if one conductor submits an unusually high number of jobs in a short window, the Security Overseer slows it down.
- **N-strike quarantine:** if a conductor has N consecutive jobs blocked, the conductor is stopped and you receive an alert.
- **Loop detection:** repeated identical jobs in a short window are flagged and blocked.
- **Resource monitoring:** abnormal process memory or CPU usage triggers an alert.

The Security Overseer runs in **active blocking mode** by default. It does not just log issues -- it prevents the jobs from executing.

5. Tool Scope Enforcement (canUseTool)

Every MCP tool call made by a task agent passes through a validation layer before it is executed. This layer checks the tool name against a per-agent allowlist.

Agent	Can use
Mail agent	Email read, email send, email search
Calendar agent	Calendar read, calendar write, event create
Files agent	Document read, document write, document search
Voice agent	Text-to-speech synthesis only

If the mail agent attempts to call a files tool, the call is blocked, logged to the audit trail, and an alert is sent. This is enforced in code, not by policy.

Why this matters: Even if a malicious email instructed the mail agent to "access my documents" or "check my calendar", the mail agent cannot comply. It physically cannot make those tool calls.

6. Message Integrity Checks

Conductors validate every inbound message before routing it to the job queue. The validation layer detects:

- **Instruction override attempts:** messages that try to change the agent's behaviour, suppress safety checks, or pretend to be from an administrator
- **Credential requests:** messages that ask the agent to reveal API keys, tokens, or passwords
- **Obfuscated commands:** encoded, fragmented, or indirectly phrased instructions designed to bypass pattern matching

On detection: the message is dropped, the attempt is logged with full context, and an alert is sent to you via the Alert Relay. The job is never written to the queue.

7. Tailscale Security Model

Your system is not exposed to the internet. The only way to access it remotely is through Tailscale, your private network.

What Tailscale does:

- Creates an encrypted peer-to-peer connection between your authorised devices
- Requires authentication to your Tailscale account to join the network
- No port forwarding required -- nothing is exposed on your public IP
- All traffic between devices is encrypted (WireGuard)

What Tailscale does NOT protect against:

- A device that is already on your Tailscale network is trusted
- If someone gains access to your Tailscale account, they can add devices to your network
- Protect your Tailscale account with a strong password and two-factor authentication

8. Certificate Trust Model

Your system uses HTTPS for all browser connections. The certificate is signed by a Certificate Authority (CA) that you created and installed on your devices.

Why a self-signed CA? Your system is not on the public internet. A certificate from a public CA (like Let's Encrypt) requires a public domain name. The self-signed CA model allows HTTPS for private, Tailscale-gated access without a public domain.

What to protect:

- The CA private key is stored at `/opt/pandoras-box/certs/ca.key` with `chmod 700`
- Do not copy the CA private key to other machines or back it up to cloud storage
- If the CA key is compromised, generate a new CA and reinstall the certificate on

all devices

9. Dependency Scanning

The Security Overseer runs a dependency scan every Thursday. The scan:

- Checks npm packages in each company's base directory against the latest versions
- Runs `npm audit` to check for known vulnerabilities
- Generates a report of packages that need updating

The report is written to `/opt/pandoras-box/argus/store/pending-mitigations.json`.

How to review and apply: When you see a "dependency scan results" alert, review the report with the System Administrator. If updates are safe to apply, the System Administrator writes a deployment script and you run it via Deploy.

10. What To Do When the Security Overseer Blocks a Job

If you see an alert that a job was blocked:

- **Do not panic.** The Security Overseer blocks anomalous jobs by design. Most blocks

are legitimate -- an unusual request that looked out of pattern.

- **Review the block reason:**

```
node -e "  
  const {DatabaseSync}=require('node:sqlite');  
  const db=new DatabaseSync('/var/ai-jobs/jobs.db');  
  const rows=db.prepare('SELECT id,company,task_type,payload,status FROM jobs WHERE  
status=\"blocked\" ORDER BY created_at DESC LIMIT 5').all();  
  console.log(JSON.stringify(rows,null,2));  
  db.close()  
"
```

- **Assess the cause:**

- If the block looks like a false positive (a legitimate request that was flagged), you can re-submit the job after reviewing it - If the block looks like a genuine concern (an email instructing the agent to do something unusual), treat it as a potential security incident

- **If you suspect a security incident:**

- Stop the affected conductor: `sudo launchctl stop com.pandoras-box.[company]-conductor` -
Review recent job history for the affected company - Check the conductor log for unusual activity -
Contact the System Administrator for a full review

11. Incident Response

If you believe your system has been compromised:

Immediate containment

Stop all conductors to prevent further actions:

```
for svc in $(launchctl list | grep pandoras-box.*conductor | awk '{print $3}'); do
    sudo launchctl stop "$svc"
done
```

Assessment

Review the Security Overseer log for the past 24 hours:

```
tail -200 /tmp/pandoras-box-argus.log
```

Review recent job queue entries:

```
node -e "
const {DatabaseSync}=require('node:sqlite');
const db=new DatabaseSync('/var/ai-jobs/jobs.db');
const rows=db.prepare('SELECT * FROM jobs ORDER BY created_at DESC LIMIT 50').all();
console.log(JSON.stringify(rows,null,2));
db.close()
"
```

Credential rotation

If any credentials may have been exposed:

- Rotate your Anthropic API keys immediately at console.anthropic.com
- Rotate your Microsoft 365 client secret at portal.azure.com
- Rotate your Google OAuth credentials at console.cloud.google.com
- Update the `.env` files with the new credentials
- Restart all services

Recovery

After assessing and containing the incident:

- Determine the root cause (was it a prompt injection? A credential exposure? A

misconfigured permission?)

- Fix the root cause before restarting conductors
- Review and update the Security Overseer's job type allowlist if a new attack pattern

was identified

- Document what happened for your own records

